Evolutionary Computing 2026

Standard Assignment – Task II

Neuroevolution for Targeted Locomotion

Group 23

Anish Behera – 2918246
Siddhant Singh Bisht – 2923456
Munishwar Pradhan – 2922549
Eduardo Martin Quintana – 2705065

13 October 2026

Small Gaussian Steps Learn Faster but Occasionally Stall in Gecko Neuroevolution

1 INTRODUCTION

In this assignment we evolve the weights of a neural-network controller so that a fixed robot body walks from its spawn position to a target position, a form of neuroevolution [8]. The body is the gecko from the John Set and the world is SimpleFlatWorld. The genotype is a vector of 138 weights, and the controller outputs target angles for the six hinges of the body. Fitness is the Euclidean distance in the horizontal (x, y) plane between the core module's final position and a target 2 m away, which we minimise; a robot that does not move scores 2.

Our EA has no crossover, so Gaussian mutation is its only source of variation, and it perturbs all weights at once. We therefore study the step size σ of this mutation, which sets how far an offspring lies from its parent in weight space and so trades exploration against exploitation [1, 4].

RQ1. How does the step size σ of Gaussian mutation affect how close an evolved gecko gets to the target? We compare σ = 0.1 with σ = 0.5 and keep every other parameter constant.

RQ2. Does σ affect how consistent the outcome is across independent runs?

H1. Because a mutation moves all 138 weights together, σ = 0.5 should change a controller too strongly for most offspring to improve on their parent (Section 2.3). We expect σ = 0.1 to improve more steadily and to reach a lower final distance. RQ2 is exploratory, and we state no hypothesis for it.

Both settings are also compared against random search at the same budget, so we can tell whether the EA does more than sample the initialisation distribution.

Both step sizes clearly beat random search: after 10,020 evaluations the mean final distance is 0.565 m for σ = 0.1 and 0.694 m for σ = 0.5, against 1.292 m. σ = 0.1 is significantly ahead early in the run because more of its offspring succeed, but the final difference between the two step sizes is not significant (p = 0.165). Its larger spread across seeds comes from a single run that stalled; the typical spread of the two settings does not differ.

2 METHODS

2.1 Task

The robot is spawned at (0.0, 0.0, 0.1) and the target is at (2.0, 0.0, 0.1). Each evaluation simulates 15 s of MuJoCo physics [10] with a 2 ms time step, run through the course fork of the ariel framework [5]. Fitness is the distance to the target in the (x, y) plane and is minimised, so a controller that leaves the robot where it started scores 2 and one that reaches the target scores 0.

2.2 Controller

The controller is a fully connected feedforward network with 17 inputs, 6 hidden units, 6 outputs and no bias terms. Both layers use tanh, and the outputs are scaled to $[-\pi/2, \pi/2]$, the range of the hinge-angle targets. The genotype is the concatenation of the two flattened weight matrices ($17 \times 6 + 6 \times 6 = 138$ values).

The network output is written directly into data.ctrl at every physics step (500 Hz), the direct option in the template, rather than accumulated as deltas. Because the scaled tanh output already lies inside the hinge range, no clipping or controller state is needed, and the command is a pure function of the current state and time.

Thirteen inputs are the values of data.qpos: core position (3), core orientation quaternion (4) and the six hinge angles. The other four are ours. Two are a beat, $\sin(2\pi ft)$ and $\cos(2\pi ft)$ with f = 1 Hz, which gives the network a periodic signal to build a gait on; single-seed tests before the final experiment showed that this input improved progress. The other two are the vector (dx, dy) from the core to the target. Because the target is fixed and the vector is expressed in world coordinates, it equals (2 − x, −y) and adds no information beyond the core position already in qpos; in a network without biases its practical effect is a constant offset input. We kept it so that the controller has an explicit target-relative signal.

2.3 Evolutionary algorithm

The EA is built on ariel.ec, using its EA, EAOperation, Individual, Population and FloatMutator classes as building blocks; the framework itself is unmodified. A population of μ = 20 individuals produces λ = 20 offspring per generation, and each weight of the initial population is drawn from $N(0, 0.5^2)$.

Parent selection is truncation: the best half of the population forms the mating pool, and each child copies a parent drawn uniformly at random, with replacement, from this pool, so a parent can have several children or none. The child is then mutated by adding Gaussian noise to every weight: each weight w_i becomes $w_i + N_i(0, \sigma^2)$, where the N_i are independent draws. The mutation probability is 1, so every weight changes.

Survivor selection is (μ + λ) [4]: parents and offspring are pooled and the best 20 survive, so the best controller is never lost. Each run lasts 500 generations, i.e. 20 initial + 20 × 500 offspring = 10,020 evaluations. We chose 500 generations because a 300-generation pilot run (seed 1) showed σ = 0.5 still improving at the end.

Motivation. We omit crossover so that σ is the only source of variation and its effect is not masked by recombination; crossover of weight vectors is also unreliable, because the same network can be encoded with its hidden units in any order [9]. Elitist truncation is a simple, strongly exploitative scheme that suits a deterministic fitness, where re-evaluation is never needed. The two step sizes are 20 % and 100 % of the initialisation scale of 0.5. Since the noise vector has an expected length of about $\sigma\sqrt{138}$, one mutation moves a genome by about 1.17 for σ = 0.1 and 5.87 for σ = 0.5. An initial genome itself has a length of about $0.5\sqrt{138} = 5.87$, and two independent initial genomes lie about 8.3 apart, so at the start a σ = 0.5 offspring is roughly 70 % as far from its parent as an unrelated random controller.

2.4 Baseline

Random search draws weight vectors from the same $N(0, 0.5^2)$ distribution and evaluates the same number of controllers (10,020) in blocks of 20; after each block we record the best distance found so far. With the same seed, its first block is identical to the EA's initial population.

2.5 Experimental setup

Table 1 lists all settings. We ran three conditions (EA with σ = 0.1, EA with σ = 0.5 and random search) ten times each with seeds 1 to 10, giving 30 runs. The same seed produces the same initial population in all three conditions, so the initial best fitness is identical across conditions for every seed (Table 4). The pilot run used seed 1, which is also a final seed; the pilot only fixed the number of generations, and no parameter was tuned on it.

We compare the final best fitness of the ten runs per condition with two-sided Mann–Whitney U tests [7] (exact p-values, α = 0.05) and report Cliff's δ [3] as effect size; a positive δ means the first condition tends to reach the lower (better)

distance. With ten runs per condition we do not assume normality. The three final comparisons are Holm-corrected [6]. Because each seed shares its initial population across conditions, we also report a paired Wilcoxon signed-rank test [11] as a robustness check. To compare the spread of outcomes (RQ2) we use the Brown–Forsythe test [2], a median-centred Levene test that is robust to outliers.

Every run is deterministic given its seed. The seed is passed on the command line and seeds both NumPy's generator and ariel.ec's package-level generator through set_seed; without the second call every seed would draw the same sequence of mutations. A single run is reproduced from the repository root (results are written under the working directory) with

```
uv run A2_template_2026.py <seed> <sigma|random>
```

for example "3 0.1" for the EA with σ = 0.1 on seed 3, or "3 random" for the baseline. We used Python 3.12 and MuJoCo 3.8.0. code/plot_results.py regenerates all figures and every statistic in this report from the 30 CSV files in data/.

Table 1: Parameter settings. Everything is identical across the two EA settings except σ.

Component	Setting
Framework / physics	ariel (course fork), MuJoCo 3.8.0, Python 3.12
Body	gecko, 6 hinges — constant
World	SimpleFlatWorld — constant
Spawn / target	(0, 0, 0.1) / (2, 0, 0.1)
Time per evaluation	15 s, 2 ms physics step
Control	direct hinge-angle targets, every step
Fitness	Euclidean (x, y) distance, minimised
Controller	17 → 6 → 6, tanh, no bias
Genotype	138 real values
Initialisation	each weight ~ $N(0, 0.5^2)$
μ / λ	20 / 20
Parent selection	truncation, best 50 %, uniform with replacement
Crossover	none
Mutation	Gaussian, p = 1.0, σ ∈ {0.1, 0.5}
Survivor selection	(μ + λ), elitist
Generations / budget	500 / 10,020 evaluations
Runs per setting	10 (seeds 1–10)
Baseline	random search, same budget
Statistics	exact Mann–Whitney U, Cliff's δ, Holm; Brown–Forsythe

3 RESULTS AND DISCUSSION

3.1 Both EA settings outperform random search

After 10,020 evaluations the EA with σ = 0.1 reaches a mean distance of 0.565 m and the EA with σ = 0.5 reaches 0.694 m, against 1.292 m for random search (Table 2). Both differences are significant with large effect sizes and survive Holm correction (Table 3). σ = 0.5 beats random search on all ten seeds and σ = 0.1 on nine; the exception is the stalled σ = 0.1 run on seed 1 (Section 3.5). The mean EA curves pass random search's final value after 16 (σ = 0.1) and 64 (σ = 0.5) generations (Figure 1). Random search does improve, from 1.797 m to 1.292 m, but only by drawing more controllers from the same distribution; it never builds on a good controller, and the mean of each fresh batch stays at about 2.0 m, the score of a robot that does not move.

Table 2: Final best fitness after 500 generations, in metres from the target. Lower is better. n = 10 runs per setting; IQR is the interquartile range.

Setting	Mean	Sd	IQR	Median	Best	Worst
EA, σ = 0.1	0.565	0.395	0.197	0.445	0.253	1.533
EA, σ = 0.5	0.694	0.214	0.263	0.792	0.300	0.899
Random search	1.292	0.090	0.153	1.318	1.156	1.388

Table 3: Two-sided exact Mann–Whitney U tests on final best fitness, n = 10 per setting, with Holm-adjusted p-values. A positive δ means the first setting reaches the lower distance more often; wins counts the seeds on which it finished lower.

Comparison	p	p_{Holm}	Cliff's δ	Wins
σ = 0.1 vs random	0.0015	0.0030	+0.80	9/10
σ = 0.5 vs random	< 0.0001	< 0.0001	+1.00	10/10
σ = 0.1 vs σ = 0.5	0.165	0.165	+0.38	7/10

Table 4: Final best fitness of every run, in metres. "Initial" is the best of the shared starting population, which is identical across the three settings for a given seed. σ = 0.1 is the lower of the two EA settings on seven of the ten seeds; the exceptions are seeds 1, 8 and 9.

Seed	Initial	σ = 0.1	σ = 0.5	Random
1	1.919	1.533	0.803	1.294
2	1.731	0.553	0.899	1.379
3	1.769	0.280	0.496	1.156
4	1.801	0.383	0.825	1.308
5	1.639	0.472	0.781	1.172
6	1.693	0.320	0.840	1.179
7	1.766	0.441	0.830	1.362
8	1.904	0.964	0.300	1.357
9	1.910	0.449	0.393	1.388
10	1.836	0.253	0.778	1.329
Mean	**1.797**	**0.565**	**0.694**	**1.292**

3.2 σ = 0.1 reaches a lower distance, but not significantly

σ = 0.1 has the lower mean (0.565 against 0.694 m) and the lower median (0.445 against 0.792 m), and it finishes below σ = 0.5 on seven of the ten seeds. The difference is not significant, however (p = 0.165, δ = +0.38; paired Wilcoxon p = 0.49), so H1 is not supported at α = 0.05, although the direction of the effect matches it.

The gap between the two step sizes is also small next to the gap to random search: changing σ by a factor of five moves the mean final distance by 0.13 m, while using an EA at all moves it by 0.60 m or more.

3.3 σ = 0.1 makes faster early progress

The two settings differ most early in the run. The gap between the mean curves peaks at 0.38 m in generation 37 (Figure 1). At generation 100, which we use as a single early checkpoint, the mean best fitness is 0.884 m for σ = 0.1 and 1.195 m for σ = 0.5 (Table 5), a significant difference (p = 0.029, δ = +0.58); generations 50 and 200 give the same picture (p = 0.002 and 0.007). By generation 500 the gap has shrunk to 0.13 m and is no longer significant.

Over the last 100 generations σ = 0.5 still improved by 0.090 m and σ = 0.1 by 0.063 m, and in the last 50 generations 3 of the ten σ = 0.5 runs and 5 of the ten σ = 0.1 runs still found a new best. Neither setting had reached the plateau that the brief asks for as a stopping criterion; we return to this in Section 3.7.

Table 5: Mean best fitness over the ten seeds at five checkpoints, in metres. Generation 0 is the shared initial population.

Generation	σ = 0.1	σ = 0.5	Random
0	1.797	1.797	1.797
100	0.884	1.195	1.424
250	0.723	0.921	1.376
400	0.627	0.785	1.327
500	0.565	0.694	1.292

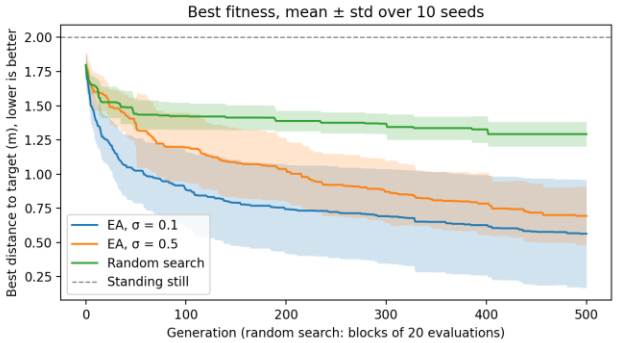


Figure 1: Best fitness per generation, mean ± standard deviation over 10 seeds per setting. For random search the horizontal axis counts blocks of 20 evaluations, so all three curves span the same budget of 10,020 evaluations. The dashed line at 2.0 marks a robot that does not move.

3.4 Step size changes how often offspring succeed

To check the mechanism behind H1, we measured how often mutation produces a useful child. Because survivor selection keeps the best 20 of parents and offspring, the population mean falls in a generation exactly when at least one child beats the worst member of the population, so the per-generation logs give the success rate of every run; we confirmed this against the full database of an additional run. In generations 1–50, at least one σ = 0.1 child entered the population in 90 % of generations, against 76 % for σ = 0.5, and σ = 0.1 found a new best in 24 % of generations against 9 % (Table 6). After generation 100 the rates converge and are no longer significantly different. Over the whole run σ = 0.1 found more new bests (22.8 against 16.9 per run), each of them smaller on average (0.054 against 0.065 m).

This supports the mechanism behind H1, but only for the first part of the run. A plausible reason it fades is weight growth. In an additional σ = 0.5 run (seed 42) whose complete database we kept, the mean genome length grew from 5.9 at initialisation to 22.5 at generation 100 and 36.7 at generation 500, while the number of surviving children fell from 2.0 to 0.4 per generation. A fixed σ = 0.5 step of length about 5.9 therefore shrinks from roughly 100 % to 16 % of the genome length, and larger weights also saturate the tanh units, which makes the network less sensitive to a perturbation of fixed size. This is a single run and an explanation rather than a test, but it is consistent with σ = 0.5 catching up late in the run.

Table 6: Mutation success, mean over ten runs per setting. A generation is successful if at least one offspring entered the population; p from exact two-sided Mann–Whitney U tests over the ten per-run values.

Measure	σ = 0.1	σ = 0.5	p
≥ 1 child survives, gens 1–50	90 %	76 %	0.003
≥ 1 child survives, gens 101–500	26 %	29 %	0.28
New best found, gens 1–50	24 %	9 %	< 0.001
New best found, gens 101–500	1.9 %	2.5 %	0.25
New bests per run	22.8	16.9	0.023
Mean gain per new best (m)	0.054	0.065	–

3.5 σ = 0.1 is not less consistent, but it can stall

Across the ten seeds the standard deviation of the final distance is 0.395 m for σ = 0.1 and 0.214 m for σ = 0.5 (Table 2). This difference comes from two runs (Figure 2): seed 1 finished at 1.533 m, worse than every random-search run, and seed 8 at 0.964 m, while the other eight σ = 0.1 runs lie between 0.253 and 0.553 m. Without seed 1 the standard deviation of σ = 0.1 drops to 0.213 m, the same as σ = 0.5, and its interquartile range is in fact smaller (0.197 against 0.263 m). The Brown–Forsythe test finds no difference in spread (p = 0.50). The answer to RQ2 is therefore that σ does not change the typical spread of the outcome, but σ = 0.1 has a heavier tail: one run in ten stalled.

The stalled run fits the usual picture of exploitation and exploration [4]. Small steps behave like local hill-climbing: they refine whatever region the initial population happens to occupy, which is fast when that region leads somewhere and slow when it does not. On seed 1, σ = 0.1 was still at 1.60 m at generation 100 and gained only 0.07 m in the remaining 400 generations, although its last improvement came as late as generation 468. From the identical initial population, σ = 0.5 reached 0.803 m, presumably because its larger steps could leave the poor region. With elitist survivor selection, a σ = 0.1 population that has converged on a mediocre controller can leave it only through rare small improvements. With ten runs, one stalled run is an observation rather than an estimate of how often this happens.

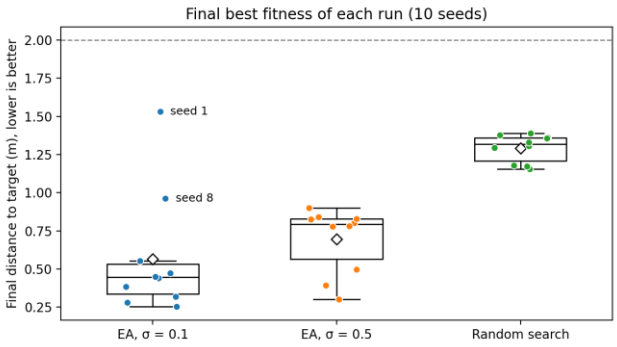


Figure 2: Final best fitness of every run (dots), with boxplots showing the median (line) and mean (diamond). Lower is better. The two weakest σ = 0.1 runs are labelled; the dashed line marks a robot that does not move.

3.6 Population convergence

At generation 500 the average gap between the population mean and the population best is 0.094 m for σ = 0.1 and 0.104 m for σ = 0.5 (Figure 3). The direction matches the exploitation reading, with the smaller step holding the population closer around one solution, but a difference of 0.010 m across ten runs is small next to the 0.2–0.4 m spread between seeds.

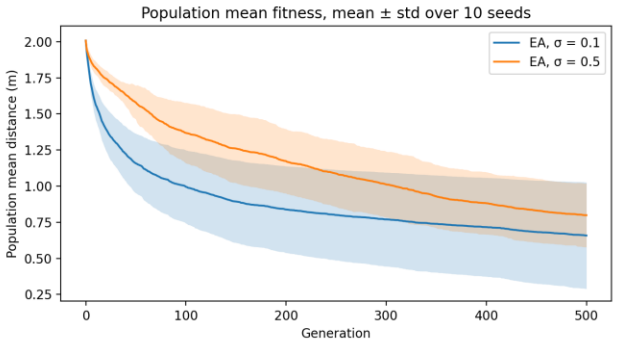


Figure 3: Population mean fitness per generation for the two EA settings, mean ± standard deviation over 10 seeds. Random search is not shown because the mean of each fresh batch stays at about 2.0 m.

3.7 Limitations

Neither setting had fully plateaued at generation 500: σ = 0.5 still improved by 0.090 m and σ = 0.1 by 0.063 m over the last 100 generations, so the comparison is truncated by the budget rather than taken at convergence, and longer runs could narrow the remaining gap further.

Ten runs are enough to separate either EA from random search but not the two EA settings from each other, mainly because of the stalled σ = 0.1 run. The weight-growth explanation in Section 3.4 rests on a single run; the per-generation logs of the 20 main runs do not store genotypes. Everything else is held constant: one body, one world, one target position, a population of 20, no crossover and two values of σ. A finer sweep of σ, or a self-adaptive step size [1], would show where between 0.1 and 0.5 the trade-off between speed and robustness lies.

Finally, fitness only measures the final distance; energy use, the path taken and whether the robot stays upright are not accounted for.

4 CONCLUSIONS

We asked how the step size of Gaussian mutation affects how close an evolved gecko gets to its target (RQ1) and how consistent the outcome is across runs (RQ2). Both EA settings clearly beat random search at an equal budget of 10,020 evaluations: 0.565 m for σ = 0.1 and 0.694 m for σ = 0.5, against 1.292 m.

For RQ1, σ = 0.1 learns significantly faster early in the run, because more of its offspring succeed, but by generation 500 the difference is not significant ($p = 0.165$), so H1 is not supported, although σ = 0.1 still has the lower mean and median and wins on seven of ten seeds. The early advantage of small steps fades as the success rates of the two settings converge, plausibly because the evolved weights grow and a fixed large step becomes relatively smaller.

For RQ2, σ does not change the typical spread of outcomes (Brown–Forsythe $p = 0.50$), but σ = 0.1 has a heavier tail: one of its ten runs stalled worse than random search. Small steps are the better choice when several runs can be made and the best kept; larger steps are the safer choice for a single run. Future work includes longer runs that reach a plateau, a finer or self-adaptive step size, and recombination.

5 USE OF AI

AI was used following the course's guidelines ("editorial support, rephrasing, or structure") to help structure this document, correct grammar and rephrase the text, and to help write and check the experiment-runner and statistical-analysis scripts. AI also suggested the additional checks in Sections 3.4 and 3.5 (offspring success rates and the spread test), which the team verified against the data. The team reviewed and ran all code, and every number reported here comes from those runs.

REFERENCES

[1] H.-G. Beyer and H.-P. Schwefel. 2002. Evolution strategies – A comprehensive introduction. *Natural Computing* 1, 1 (2002), 3–52.

[2] M. B. Brown and A. B. Forsythe. 1974. Robust tests for the equality of variances. *Journal of the American Statistical Association* 69, 346 (1974), 364–367.

[3] N. Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509.

[4] A. E. Eiben and J. E. Smith. 2015. *Introduction to Evolutionary Computing* (2nd ed.). Springer, Berlin, Heidelberg.

[5] Evolutionary Computing 2026. 2026. Course fork of the ariel framework. GitHub repository. https://github.com/AndrzejSzczepura/EvolutionaryComputing2026

[6] S. Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[7] H. B. Mann and D. R. Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60.

[8] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen. 2019. Designing neural networks through neuroevolution. *Nature Machine Intelligence* 1, 1 (2019), 24–35.

[9] K. O. Stanley and R. Miikkulainen. 2002. Evolving neural networks through augmenting topologies. *Evolutionary Computation* 10, 2 (2002), 99–127.

[10] E. Todorov, T. Erez, and Y. Tassa. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 5026–5033.

[11] F. Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.